

1) scatter plot and hill climbing

```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
plt.scatter(X[y==0, 0], X[y==0, 1], color='blue', label='Setosa')
plt.scatter(X[y==1, 0], X[y==1, 1], color='red', label='Versicolor')
```

```
plt.title("Iris Scatter Plot")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.legend()
plt.show()
```

Hill climbing:

```
def hill_climb(start, step_size, max_iterations):
    def fitness(x):
        return -x**2 + 4 # The function we want to maximize

    current = start
    for _ in range(max_iterations):
        neighbor = current + step_size
        if fitness(neighbor) > fitness(current):
            current = neighbor
        else:
            break # No better neighbor found
    return current, fitness(current)

solution, value = hill_climb(start=2, step_size=-0.1, max_iterations=100)
print("Best solution:", solution)
print("Best value:", value)
```

2) 3d plot and BFS

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from mpl_toolkits.mplot3d import Axes3D

# Load iris dataset
iris = load_iris()
X = iris.data
y = iris.target # species labels

# 3D scatter plot using first 3 features
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=y, cmap='viridis')

ax.set_title("3D Scatter Plot - Iris Dataset")
ax.set_xlabel("Sepal length (cm)")
ax.set_ylabel("Sepal width (cm)")
ax.set_zlabel("Petal length (cm)")
plt.show()
```

BFS

```
from collections import deque

# BFS function
def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            queue.extend(neighbor for neighbor in graph[node] if neighbor not in visited)
            # for neighbour in graph[node]:
            #     If neighbour not in visited:
            #         queue.append(neighbour)

# Example graph (adjacency list)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```

'C': ['F'],
'D': [],
'E': ['F'],
'F': []
}

# Call BFS
bfs(graph, 'A')

```

```

class tree_node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

node = tree_node(10)
node.left = tree_node(5)
node.right = tree_node(15)
node.left.left = tree_node(3)
node.left.right = tree_node(4)
#output = 10 , 5 , 15 , 3 , 4

from collections import deque
def bfs(node):
    queue = deque([node])
    visited = []
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node.data)
            visited.append(node)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

bfs(node)

```

3) contour plot and A*

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_iris

# Load iris dataset
iris = load_iris()
X = iris.data[:, 0] # sepal length
Y = iris.data[:, 1] # sepal width

# Make Z as a simple function of X and Y
Z = np.sin(X) + np.cos(Y)

# Create contour plot
plt.tricontourf(X, Y, Z, cmap="plasma") # contour from scattered points
plt.colorbar()
plt.title("Contour Plot - Iris Dataset (Sepal length vs Sepal width)")
plt.xlabel("Sepal length (cm)")
plt.ylabel("Sepal width (cm)")
plt.show()
```

A*

```
import heapq

# A* Search on Tree
def astar_tree(graph, start, goal, heuristic):
    # Priority queue: (f, g, node, path)
    open_list = [(heuristic[start], 0, start, [start])]

    while open_list:
        f, g, node, path = heapq.heappop(open_list)

        if node == goal:
            return path, g # return path and cost

        for neighbor, cost in graph.get(node, []):
            new_g = g + cost
            new_f = new_g + heuristic[neighbor]
            heapq.heappush(open_list, (new_f, new_g, neighbor, path + [neighbor]))

    return None, float('inf')

# Example Tree (adjacency list: node -> [(child, cost), ...])
graph = {
    'A': [('B', 1), ('C', 4)],
```

```

    'B': [('D', 5), ('E', 2)],
    'C': [('F', 1)],
    'E': [('G', 3)]
}

# Heuristic values (estimated cost to reach goal 'G')
heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 99, # large (far from goal)
    'E': 1,
    'F': 4,
    'G': 0 # goal has 0
}

start, goal = 'A', 'G'
path, cost = astar_tree(graph, start, goal, heuristic)
print("Path:", path)
print("Total cost:", cost)

```

4) Heat map and Min-max algorithm

```

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

# Load iris dataset
iris = load_iris()
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Correlation matrix heatmap
sns.heatmap(iris_df.corr(), annot=True, cmap="viridis")
plt.title("Heatmap of Iris Dataset (Feature Correlations)")
plt.show()

```

min –max

Minimax Algorithm Implementation for a Game Tree

```
def minimax(depth, node_index, is_maximizing_player, scores, height):
```

```
    # Base case: leaf node
```

```
    if depth == height:
```

```
        return scores[node_index]
```

```
    if is_maximizing_player:
```

```
        return max(
```

```
            minimax(depth + 1, node_index * 2, False, scores, height),
```

```
            minimax(depth + 1, node_index * 2 + 1, False, scores, height)
```

```
        )
```

```
    else:
```

```
        return min(
```

```
            minimax(depth + 1, node_index * 2, True, scores, height),
```

```
            minimax(depth + 1, node_index * 2 + 1, True, scores, height)
```

```
        )
```

```
# Example leaf node scores
```

```
scores = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
# Height of the tree = log2(len(scores)) = 3
```

```
tree_height = 3
```

```
# Call minimax starting from root (depth=0), root index=0, maximizing player's turn
```

```
best_value = minimax(0, 0, True, scores, tree_height)
```

```
print("The optimal value is:", best_value)
```

5) Box plot and Alpha beta pruning

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load iris dataset
```

```
iris = sns.load_dataset("iris")
```

```
# Simple boxplot for all numeric features
```

```
sns.boxplot(data=iris)
```

```
plt.title("Box Plot - Iris Dataset")
```

```
plt.xlabel("Features")
```

```
plt.ylabel("Values")
```

```
plt.show()
```

Alpha beta

```
def alphabeta(depth, node_index, is_max, scores, alpha, beta, height):
    if depth == height:
        return scores[node_index]

    if is_max:
        best = float('-inf')
        # Explore left and right child
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, False, scores, alpha, beta, height)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break #  $\beta$  cut-off
        return best
    else:
        best = float('inf')
        # Explore left and right child
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, True, scores, alpha, beta, height)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break #  $\alpha$  cut-off
        return best

# Example leaf node values
scores = [3, 5, 6, 9, 1, 2, 0, -1]

# Tree height = 3 (log2 of 8 leaf nodes)
tree_height = 3

# Start at root: depth=0, node_index=0, maximizing=True
optimal = alphabeta(0, 0, True, scores, float('-inf'), float('inf'), tree_height)
print("The optimal value is:", optimal)
```

6) Navy base classifier

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load Titanic dataset from CSV
# Replace with your actual path if needed
data = pd.read_csv('titanic.csv')

# Select features and target (you can customize as needed)
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
target = 'Survived'

# Handle missing values (simple strategy)
data = data[features + [target]].dropna()

# Encode categorical columns
le = LabelEncoder()
data['Sex'] = le.fit_transform(data['Sex']) # male=1, female=0

# Split into input (X) and output (y)
X = data[features]
y = data[target]

# Split into training and testing (70-30)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Train Naive Bayes classifier
model = GaussianNB()
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print("Accuracy of Naive Bayes classifier:", accuracy_score(y_test, y_pred))
```


7) KNN

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the glass dataset from local CSV file
# Replace 'glass.csv' with the correct filename or path if needed
data = pd.read_csv('glass.csv')

# Separate features (X) and target (y)
X = data.iloc[:, :-1] # All columns except the last (features)
y = data.iloc[:, -1] # Last column (target class)

# Split dataset into 70% training and 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Standardize the feature values
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN with Euclidean distance (default)
knn_euclidean = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn_euclidean.fit(X_train, y_train)
y_pred_euclidean = knn_euclidean.predict(X_test)
print("Accuracy with Euclidean distance:", accuracy_score(y_test, y_pred_euclidean))

# KNN with Manhattan distance
knn_manhattan = KNeighborsClassifier(n_neighbors=3, metric='manhattan')
knn_manhattan.fit(X_train, y_train)
y_pred_manhattan = knn_manhattan.predict(X_test)
print("Accuracy with Manhattan distance:", accuracy_score(y_test, y_pred_manhattan))
```

8) K means

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load and scale data
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)

# Apply K-Means (3 clusters for 3 Iris species)
kmeans = KMeans(n_clusters=3, random_state=0)
labels = kmeans.fit_predict(X)

# Scatter plot using first two features
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("K-Means Clustering (Iris Dataset)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

9) single and complete linkage

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage

# Load and standardize the Iris dataset
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)

# Function to plot dendrogram
def plot_dendrogram(X, method):
    Z = linkage(X, method=method)
    plt.figure(figsize=(6, 4))
    plt.title(f'Dendrogram - {method} linkage')
    dendrogram(Z)
    plt.xlabel("Samples")
    plt.ylabel("Distance")
    plt.tight_layout()
    plt.show()

# Plot dendrograms for both linkage methods
for method in ['single', 'complete']:
    plot_dendrogram(X, method)
```

11)PCA and LDA

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
import matplotlib.pyplot as plt
```

```
# Load the Iris dataset
iris = load_iris()
X = iris.data    # Features (4 columns)
y = iris.target  # Labels (0 = setosa, 1 = versicolor, 2 = virginica)

print("Shape of original data:", X.shape)
# -----
# PCA (Principal Component Analysis)
# -----
print("\n--- PCA ---")
pca = PCA(n_components=2)    # Reduce to 2 dimensions
X_pca = pca.fit_transform(X)

print("Shape after PCA:", X_pca.shape)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Plot PCA result
plt.figure(figsize=(6, 4))
plt.title("PCA - Iris Dataset")
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()

# -----
# LDA (Linear Discriminant Analysis)
# -----
print("\n--- LDA ---")
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X, y)

print("Shape after LDA:", X_lda.shape)

# Plot LDA result
plt.figure(figsize=(6, 4))
plt.title("LDA - Iris Dataset")
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='plasma')
plt.xlabel("LDA 1")
plt.ylabel("LDA 2")
plt.show()
```

12) Write a Program to develop simple single layer perceptron to implement AND, OR Boolean functions.

```
# Step function
def step(x):
    return 1 if x >= 0 else 0

# Training function
def train(X, y):
    w1, w2 = 0, 0 # weights
    b = 0 # bias
    lr = 1 # learning rate

    for _ in range(10): # 10 iterations
        for i in range(4): # for all 4 input examples
            x1, x2 = X[i]
            z = w1 * x1 + w2 * x2 + b
            y_pred = step(z)
            error = y[i] - y_pred
            # update weights and bias
            w1 += lr * error * x1
            w2 += lr * error * x2
            b += lr * error

    return w1, w2, b

# Prediction
def predict(X, w1, w2, b):
    results = []
    for x1, x2 in X:
        z = w1 * x1 + w2 * x2 + b
        results.append(step(z))
    return results

# Input and Output for AND
X = [[0,0], [0,1], [1,0], [1,1]]
y = [0, 0, 0, 1]

# Train and predict
w1, w2, b = train(X, y)
print("AND Gate Output:", predict(X, w1, w2, b))
```

output:

AND Predictions: [0, 0, 0, 1]

OR Predictions: [0, 1, 1, 1]